Analysis of 'When Malware is Packin' Heat; Limits of Machine Learning Classifiers Based on Static Analysis Features'

Thomas Turner
M14507785

Nathaniel Hafely
M14560068

1 INTRODUCTION

With the rise of machine learning and artificual intelligence in recent years, many malware analysis experts have begun to research how these new tools can be used in their field. Many papers have begun to be published on the topic of using machine learning classifiers to distinguish between packed benign and packed malicious samples. The paper that we analyzed aims to verify the accuracy of these tests and see for themselves if machine learning can indeed be used to distinguish the maliciousness of packed samples based on static features alone [1].

Creating tools to automatically detect if a sample is malicious or not is very sought after in the malware field. Currently the best and most effective way to determine if a sample is malicious is to have an expert analyzing it using various static and dynamic analysis tools. This process entails placing the malware in an isolated environment and analyuzing the static features such as imports, strings, and assembly code without running the code. Then the analyst will run debug the code to see any registry key changes, file system changes, or network connections the sample is attempting to make. This process can be very tedious and can take an expert a long time to complete.

There are also many online tools now such as VirusTotal that be used to automatically scan a sample to determine if it is malicious or not. But, this method also has many downsides. VirusTotal works by using signature-based detection methods to determine if a sample is malicious. So, if the signature of a sample is similar to a known malware, then VirusTotal will detect that the given sample is potentially malicious [1]. This technique is effective at detecting previosuly known malware, but will struggle on types of malware that it has never encoutnered before.

VirusTotal is also known to struggle with packed samples. The paper mentions a study that was conducted by Lakshaman Nataraj. In this study, they collected a large number of benign Windows executables and then packed them with with four different common packers. Next, they submitted all of these samples to VirusTotal. From this experiemnt, they found that 96.7 percent of the samples caused ten or more malicious detections on VirusTotal [1]. This shows that VirusTotal also uses packing as a strong indicator of malciousness instead of the actual functionality of a sample.

Nowadays, many benign samples are also packed to protect the authors from getting their intellectual property stole [1]. Therefore, it is no longer the normal for exclusively malicious samples to be packed. Figure 1 below shows the results of a brief experiment that the researchers conducted. In this experiemnt, they collected a large number of samples from the real world and then analyzed them to see how many of them were packed based on each cateogry of benign, malicious, and suspicious [1]. The results show that in the worst quarters around 50% of the benign samples that they collected were packed [1].

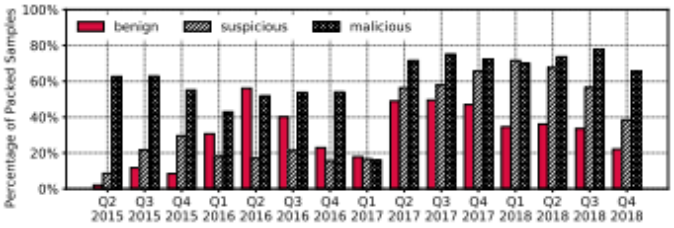

Figure 1: Prevalence of packed samples in the wild.

Through their experiments, the researchers show that packing routines do perserve some information on the function and maliciousness of a sample, but it is not enough to be applied to packers and packing routines that the classifier has never seen. Additionally, the classifier is susceptible to adversarial attacks [1]. This means that if malware authors can place benign code in their malware to try and confuse the classifier into thinking that the entire program is benign.

2 SYSTEM/THREAT MODEL

For their experiments, the researchers used samples exclusiuvely from the Windows x86 operating system. The samples can be broken up into four unique categories: unpacked benign, packed benign, unpacked malicious, and packed malicious [1]. For each individual experiment, the data was broken up into a training set and a testing set. The type of classiifer that was used for the analysis was a random forest model. Additionally, the classifier was trained on nine different sets of features in the samples (PE Headers, PE Sections, DLL Imports, API Imports, Rich Header, Byte n-grams, Opcode n-grams, Strings, File Generic) [1].

3 METHODOLOGY

For the researchers to be able to accurately train a machine learning classifier to distinguish between a benign or malicious sample, they need lots of accurate data and independent features of each piece of data. The researchers also broke up the experiments into four different categories based upon what research question the results were answering.

ACM XXXXX'18, XX 2018, XX, XX, XX

ACM ISBN 123-4567-24-567/08/06…$15.00
https://doi.org/10.475/123_4

3.1 Building the Datasets

The researchers began building the dataset by collecting data from a commercial anti-malware vendor and a labeled dataset called EMBER. The data from these two sources was then placed into the four categores of unpacked benign, packed benign, unpacked malicious, and packed malicious [1]. From the commercial anti-malware vendor, they collected a total of 29,573 executables. All of the samples from this source were pre-labeled, so no further analysis had to be done to place these samples into the correct cateogry. From the EMBER dataset, they collected a total of 800,000 executables. But, this dataset was only labeled between if the sample was benign or malicious. So, 56,411 of the samples were taken and then entered into the commercial anti-malware vendor to further categorize them into either packed or unpacked. As seen below in figure 2, the the data from these two sources totals to 85,984 samples [1].

Samples' Origin	Malicious/Benign Label's Source				# of Samples
	VirusTotal	Comm. Anti-mal.	EMBER		
(1) Comm. Anti-malw.	A	A	N/A		29,573
	A	B	N/A		15,736
	B	B	N/A		**13,046**
	A	M	N/A		13,837
	M	M	N/A		**13,536**
(2) EMBER	A	A	A		56,411
	A	A	B		24,348
	A	B	B		24,225
	B	B	B		**24,223**
	A	A	M		32,063
	A	M	M		31,087
	M	M	M		**31,066**
(1) ∪ (2)	A	A	A		**85,984**
	B	B	B		**37,269**
	M	M	M		**44,602**

Figure 2: Raw dataset collected from the commercial anti-malware vendor and the EMBER dataset.

This data was then used to build the Wild Dataset. As the name suggests, this dataset is meant to be a representation of samples in the real world. This dataset was built by verifying all of the raw data that was collected to ensure that the labels were correct. To determine if the benign/malcious labels were correct, the researchers cross-referenced all of the samples to between VirusTotal, the anti-malware vendor, and the EMBER dataset. If there were any discrepancies between these three sources, then those samples were discarded. Using this method, a total of 4,113 samples were discarded [1]. To verify that the packed/unpacked labels were accurate, the researchers compared all of the samples against the anti-malware vendor, Deep Packer Inspector (dpi), and various other malware tools (as can be seen in Figure 3). From this analysis, 31,147 samples were discarded [1]. This narrowed down wild dataset to a total of 50,724 samples.

Next, the researchers built a separate dataset called Lab Dataset. This dataset consisted of all of the samples in the Wild Dataset, but packed by nine different known packers. Some of the samples were not able to be packed by all of the packers, so those samples were discarded [1]. This resulted in the Lab Dataset having a total of 341,444 samples (shown in Figure 4) [1].

Tool	Benign		Malicious	
	packed	unpacked	packed	unpacked
(1) vendor's sandbox	10,463	16,162	26,699	15,095
(2) *dpi*	6,049	20,576	27,995	13,799
(3) *Manalyze*	1,239	17,436	5,376	19,457
(4) *PEiD+F-Prot*	1,189	25,436	2,630	39,164
(5) *yara*	1,524	25,101	3,882	37,912
(6) *Exeinfo PE*	1,088	25,537	5,770	36,024
(1)+(2)+(3)+(4)+(5)+(6)	12,647	4,396	33,681	5,752

Figure 3: Packers Used to Verify the Packed/Unpacked Labels on Wild Dataset.

Packer	# Benign Samples	# Malicious Samples	Keeps Rich Header?	# Invalid Opcodes
Obsidium	16,940	31,492	29.82%	0
Themida	15,895	26,908	✓	0
PECompact	5,610	28,346	✓	723
Petite	13,638	25,857	✓	318
UPX	9,938	20,620	✓	0
kkrunchy	6,811	15,494	19.68%	61
MPRESS	11,041	11,494	19.84%	629
tElock	5,235	30,049	✓	8
PELock	6,879	8,474	20.60%	461
AES-Encrypter	17,042	33,681	✗	0

Figure 4: Packers Used to Build the Lab Dataset.

3.2 Identifying the Features

To accurately train a classifier to determine between benign or malicious samples, you need good features to train the classifier on. The researchers identified nine different sections of the samples to grab the features from. These sections were PE Headers (28 features), PE Sections (570 features), DLL Imports (4,305 features), API Imports (19,168 features), Rich Header (66 features), Byte n-grams (13,000 features), Opcode n-grams (2,500 features), Strings (16,900 features), and File Generic (small number of features) [1]. All of these sections represent important aspects of the samples that can be used to decipher some of the functionality of the program.

4 EXPERIMENT RESULTS AND ANALYSIS

Does static analysis on packed binaries provide rich enough features to a malware classifier [1]? In order to help answer this question, the researchers determined a subset of questions that they then designed experiments to answer.

4.1 Effects of Packing Distribution During Training

Does a bias in the distribution of packers used in benign and malicious samples cause the classifier to learn specific packing routines as a sign of maliciousness [1]? To answer this question, the researchers designed three experiments.

Experiment I: "no packed benign". Using the Wild Dataset the classifier was trained on 3,956 packed malicious executables and 3,956 unpacked benign executables. The classifier was trained down to a false negative rate of 3.82% and 2.64% but when tested on 12,647 packed benign samples, this produced a false positive rate of 23.40%.

This shows that the classifier has a bias towards using packing as an indicator to distinguish if a sample is malicious.

Experiment II: "packer classifier". The Lab Dataset was used to create a classifier to distinguish between the nine different packers that were used when creating the this dataset. The classifier was trained on 107,471 samples that had an even distribution of each of the nine packers. When tested on a testing set of 46,059 samples, it was able to maintain a precision and recall percent of 99.99% for each class. This shows that if samples from other studies did not have a lot of overlap in terms of packers on both their benign and malicious samples, then the classifier could have just been biased towards certain packers or packing routines.

Experiment III: "good-bad packers". To test the question that the previous experiment raised, the classifier was then trained on a new dataset where the benign samples were packed by four packers and the malicious samples were packed by the remaining five packers. The accuracy of this classifier was quite low and varied anywhere between 0.01% and 12.57%. This shows that the classifier is heavily biased to try and distinguish the maliciousness of a sample by the packer or packing routine that is used to pack that sample.

Through this set of experiments, the researchers determined that the lack of overlap between packers used in benign and malicious samples will bias the classifier towards distinguishing between packing routines [1].

4.2 Packers vs. Malware Classification

Do packers prevent machine-learning-based malware classifiers that leverage only static analysis features [1]? To answer this question, the researchers designed three experiments.

Experiment IV: "different packed ratios (wild)". In this experiment, the classifier was trained on a subset that included only packed benign, unpacked benign, and packed malicious samples. The classifier was trained on increasing ratios of packed benign samples. The results show that as the number of packed benign samples increased, then the false positive rate on packed samples decreased, but the false positive rate on unpacked benign samples increased from 3.18% to 16.24%. This shows that a classifier that is only trained on packed samples will not be able to accurately predict the maliciousness of unpacked samples.

Experiment V: "different packed ratios (lab)". This experiment was a repeat of the previous experiment but using the lab dataset combined with unpacked benign executables from the wild dataset. Increase in the ratio of packed benign samples decreased false positive rates for packed executables from 99.76% to 16.03%. This shows that classifiers without a high ratio of benign samples rely on detecting packing rather than malicious behavior.

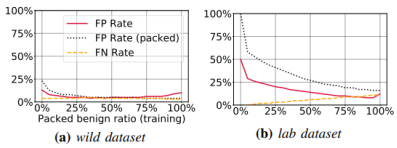

Figure 5: Different packed ratios.

Experiment VI: "single packer". This experiment was to test if classifiers are detecting malicious behavior or specific packers. To do this the researchers trained and tested each packer individually. To determine how packers are preserving information, the researchers created a different model for each family of features. Each family was found to have relatively good success but, varied amongst different packers.

Through this set of experiments, the researchers determined that packers preserve some information when packing programs that may be "useful" for malware classification, however, such information does not necessarily represent the real nature of samples [1].

4.3 Malware Classification in Real-world Scenarios

Can a classifier that is carefully trained and not biased towards specific packing routines perform well in real-world scenarios [1]? To answer this question, the researchers split the question into three main problems.

4.3.1 Generalization. Classifiers may have poor performance against previously unseen packers. To explore this, the researchers designed three experiments.

Experiment VII: "wild vs. packers". Trained the classifier on a dataset with a "wild ratio" of packed benign samples extracted from the wild dataset, and tested it on the lab dataset. This performed poorly against all packers. This is because, the classifier chose features with more information gain but, while testing on the lab dataset, those features are no longer useful. Training with only the rich header brought accuracy up to over 90%.

Experiment VIII: "withheld packer". This experiment involved several rounds of experiments on the lab dataset, withholding one packer from the training set and then evaluating the resulting classifier on packed executables generated by this packer. Excluding PECompact, tElock, and kkrunchy, there was relatively good performance. Byte n-grams features were withheld and directly responsible for the poor performance of these exceptions.

Experiment IX: "lab against wild". Trained on lab dataset and evaluated on packed executables in wild dataset. Benign and malicious executables were uniformly selected from packers to avoid bias. This had false negative and false positive rates of 41.84% 7.27% respectively.

The researchers determined that, based on the results of these three experiments, when using static analysis features, the classifier is not guaranteed to generalize well to previously unseen packers [1].

4.3.2 Strong & Complete Encryption. The static features that these classifiers rely on could be removed during the packing process by malware authors. Despite this, can malware classifiers still be effective? To answer this question, the researches designed one experiment.

Experiment X: "Strong & Complete Encryption". Trained and tested on executables packed with AES-Encrypter. There was an inflated accuracy due to file size, file entropy, and imbalanced dataset but,

after fixing these issues the accuracy dropped from 72.67% to 50%. The researchers determined that when packing hides all information about the original binary until execution, the classifier has no choice but to classify any sample packed by such a packer as malicious [1].

4.3.3 Adversarial Examples. Due to the potential vulnerability of adversarial examples, the researches as if it is possible to use the learned model to drive evasion? To answer this question, the researches designed one experiment.

Experiment XI: "adversarial samples". The classifier was trained on 3,956 unpacked benign, 3,956 packed benign, and 7,912 malicious executables. After training, the researchers generated adversarial samples from all 2,494 true positives by identifying byte n-gram and string features that occurred more in benign samples and injecting the corresponding bytes into the target program without affecting its behavior. The researchers determined that a more complex attack is needed when the classifier is trained using features extracted from dynamic analysis, which represent the sample's behavior [1].

After investigating these three main problems, the researchers determined that although static analysis features combined with machine learning can distinguish between packed benign and packed malicious samples, such a classifier will produce intolerable errors in real-world settings [1].

4.4 Anti-malware Industry vs. Packers

How is the accuracy of real-world anti-malware engines that leverage machine learning combined with static analysis features affected by packers [1]? To answer this question, the researchers designed one experiment.

Experiment XII: "anti-malware industry". 6,000 benign and 6,000 malicious executables packed with each packer from the lab dataset were sent to VirusTotal to evaluate six products on VirusTotal labeled as machine-learning-based malware detectors. Through this experiment, the researchers determined that machine-learning-based anti-malware engines on VirusTotal detect packing instead of maliciousness [1].

5 CONCLUSION AND DISCUSSION

In this paper, the researchers investigated if static analysis on packed binaries provide a rich enough set of features to a malware classifier [1]? For further investigation, the researchers split the question in three and determined the following: The lack of overlap between packers used in benign and malicious samples will bias the classifier toward distinguishing between packing routines [1]. Packers preserve some information when packing programs that may be "useful" for malware classification, however, such information does not necessarily represent the real nature of samples [1]. Although static analysis features combined with machine learning can distinguish between packed benign and packed malicious samples, such a classifier will produce intolerable errors in real-world settings [1]. Overall, the researchers determined that all the discovered issues suggest that malware detection should be done using a hybrid approach leveraging both static and dynamic analysis [1].

This is the Conclusion and Discussion Section.

REFERENCES

[1] Hojjat Aghakhani and Fabio Gritti and Francesco Mecca and Martina Lindorfer and Stefano Ortolani and Davide Balzarotti and Giovanni Vigna and Christipher Kruegel. 2020. When Malware is Packin' Heat; Limits of Machine Learning Classifiers Based on Static Analysis Features. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.